

Distributed Order Management: Business & Technical Summary

Why DOM is a combinatorial optimization problem, and what advanced methods buy

1. What the problem is

Every day, Nestlé planners meet orders that their default distribution center (DC) cannot fill completely. Stock runs out, a dock is fully booked, or a pick line is at its limit. The planner then has two choices: leave the order and send a partial delivery, which loses sales and can trigger a penalty; or move it to another DC that has the stock, and pay more to ship it.

Distributed Order Management is that choice, made for a whole order book at once. This data pack has 1,109 open orders, 1,110 SKUs and 8 DCs; 472 orders need a decision. The objective balances three things:

$$\text{Objective} = \text{Sales revenue} - \text{Penalty for unmet demand} - \text{Shipping cost}$$

2. Why it is a combinatorial optimization problem

It looks like each order can be handled on its own: find the DC that fills it best and move it there. That fails, because the orders compete with each other.

- **Stock is shared.** Two orders that need the same SKU at the same DC cannot both have it. Whoever is decided first takes it away from the other.
- **Capacity is shared.** A DC has a fixed number of dock slots and a fixed amount of picking work per day. Every move uses some of it.
- **Moves connect to each other.** Moving one order out of a DC frees stock, which can let a second order move in. You only see this if you look at both orders together.

So the decisions depend on each other, and the number of combinations grows as (candidate DCs) to the power of (orders). With 472 orders and 2,295 candidate assignments you cannot try them all. This is a constrained assignment problem, NP-hard in general, and exactly the kind where optimization pays.

Our results show this coupling is real. Under the same rules, the solver and the heuristic disagree on most orders: 37 are moved by the solver only, 12 by the heuristic only, and just 26 by both to the same DC.

3. What we built, and what it delivered

Method	Objective (\$)	Fill rate	Orders moved	Runtime
Do nothing (default assignment)	44,365,994	90.47%	0	0.21s
Greedy heuristic (by order value)	44,810,604	91.28%	41	0.40s
Exact optimization (MILP)	46,295,828	93.28%	66	63.72s

Table 1. Measured on the 472 focus orders. All methods use the same objective and the same feasibility checks.

Exact optimization is worth \$1,929,835 more than doing nothing. It also pays \$15,285 *less* in extra freight than the cheaper heuristic, even though it moves more orders. Planning all orders together lets it pick better destinations and connect moves.

4. The three trade-offs that matter

4.1 Solution quality against runtime

Doing nothing is instant. The greedy heuristic takes 0.4 seconds and gains \$444,610. The exact solver takes 63.7 seconds and gains \$1,929,835. For this problem the exact solver clearly wins: one minute of computing is nothing next to a planning cycle of hours, and the money is four times larger. The heuristic still has a use — a quick answer when a planner changes one input during the day, or a starting point for the solver.

4.2 Exact solvers against heuristics

The real difference is not speed but knowledge. A heuristic gives an answer; an exact solver gives an answer *and* a proof that nothing better exists. Without that proof you cannot know how much you are missing, so you cannot tell whether a better method is worth building.

This gap grows with problem size. We measured how far the heuristic falls below the proven best answer at five sizes:

Focus orders	50	100	200	300	472
Heuristic gap	0.84%	2.08%	3.11%	3.12%	3.21%

On small problems a heuristic is almost as good as the best answer. The more orders compete for the same stock, the more it loses, because it never goes back to change an earlier decision. A full production order book would make this gap larger.

Heuristics keep three real advantages: a planner can follow them step by step, they always return something, and they slow down gently rather than failing.

4.3 Noisy quantum hardware against simulators

We built the QUBO/Ising version of the model, where the assignment and capacity rules move into the objective as penalty terms. Our encoding keeps *only* assignment variables and adds no slack variables, so a batch of 6 orders against 3 DCs needs 18 qubits — about a third of what a slack-based encoding of the same batch would need.

We validated it against ground truth before scaling. On a 3×3 instance with 27 feasible states we enumerated every one. Ranking them by QUBO energy reproduced their ranking by business value exactly, and the ground state equalled the true optimum of \$760,518. Five independent warm-start CVaR-QAOA runs gave a mean of \$760,341 with a spread of 0.046%; four hit the optimum exactly and all five beat doing nothing.

Scaled to 15 batches covering 75 high-value orders, it raised the objective by \$1,358,671 (+10.0%) and the fill rate from 88.6% to 97.3%, with zero rule violations and 1.07 seconds of total solve time. It matched or beat the classical heuristic on all 15 batches. A hybrid variant, where QAOA selects among generated columns, closed every one of its 5 batches with a gap of exactly zero.

The equal result is the honest headline. At 18 qubits a batch is small enough that both methods find the same best answer, so this shows the encoding is correct, not that quantum is faster. What it does establish are the properties that must hold before any hardware advantage could be claimed: a faithful energy ordering, a ground state equal to the true optimum, stable repeated runs, no slack qubits, and a batching scheme that keeps qubit count flat as orders grow.

All of this used exact diagonalisation and noise-free simulation. Real hardware would add gate errors and decoherence, and this formulation is sensitive to them, because a broken rule is only discouraged by a weight and never forbidden. The large one-hot weight that guarantees correctness in simulation also widens the energy range a device must resolve — the practical trade-off in QUBO logistics models. Batching avoids the hardware limit but discards the links between batches, and those links are exactly what gave the classical solver its advantage.

5. What we would do next

- 1. Start the solver from the heuristic answer.** The heuristic takes 0.4 seconds and already finds 23% of the value. Giving that answer to the solver as a starting point cuts the search time on larger order books.
- 2. Look at which DCs carry which SKUs.** A missing SKU at the alternate DC blocks 40% of all candidate moves — the second largest cause after the divert gate. Widening the assortment at a few DCs would open moves no solver can create today.
- 3. Revisit the 5% divert gate.** It rejects 46% of all candidate moves — more than any physical constraint — and is a policy parameter, not a law of the warehouse.